ACCENTURE — USO INTERNO

Claude Registry
Guida Operativa

Installazione, utilizzo e contribuzione delle capability condivise di Claude Code

---

Versione 2.0 — Aprile 2026
Include: Architettura Skill Layer
12 Agenti · 4 Skill

Accenture

Indice

Parte 1 — Usare le Capability

1. Prerequisiti 3
2. Installazione con setup-capabilities.sh 4
3. Verifica dell'installazione 6
4. Aggiornare e personalizzare le capability 7

Parte 2 — Contribuire al Registry

5. Struttura del repository 8
6. Creare un nuovo agente 9
7. Creare una nuova skill 11
8. Scrivere il system prompt 13
9. Testare localmente 15
10. Aprire una PR e gestire la review 16
11. Pubblicare nel marketplace 17

Versione 2.0 — Novità principali: questa guida documenta il nuovo **skill layer**, introdotto per eliminare la duplicazione di standard tra gli agenti. Le skill sono knowledge provider condivisi (es. `java-spring-standards`, `testing-standards`) che vengono installati automaticamente come dipendenze. I consumatori non devono fare nulla di diverso rispetto alla versione 1.x.

PARTE 1

Usare le Capability

1. Prerequisiti

Prima di installare le capability di Claude Registry nel tuo progetto, verifica che l'ambiente sia configurato correttamente.

1.1 Strumenti richiesti

Strumento	Versione minima	Verifica
Claude Code	Versione corrente (CLI Anthropic)	`claude --version`
Python 3	3.9+	`python3 --version`
Git	2.x	`git --version`
jq (opzionale)	1.6+	`jq --version`

1.2 Accesso al repository

Il registry è ospitato su Git interno. Clona il repository prima di procedere:

```
git clone <repo-url> ~/dev/claude-registry
cd ~/dev/claude-registry
```

1.3 Struttura del progetto destinatario

Le capability vengono installate nella directory `.claude/agents/` del tuo progetto. La directory viene creata automaticamente dallo script se non esiste.

```
tuo-progetto/
  .claude/
    agents/           ← le capability vengono copiate qui
    settings.json     ← aggiornato automaticamente con i permessi Agent()
```

> **Nota:** Claude Code rileva automaticamente i subagent nella directory `.claude/agents/`. Non è necessaria nessuna configurazione aggiuntiva oltre allo script di installazione.

2. Installazione con setup-capabilities.sh

Lo script interattivo è il metodo consigliato. Gestisce selezione, copia dei file, risoluzione delle dipendenze (skill) e aggiornamento di `settings.json` in un'unica operazione.

2.1 Esecuzione base

```
# Installa nel progetto corrente
./claude-catalog/scripts/setup-capabilities.sh

# Installa in un progetto specifico
./claude-catalog/scripts/setup-capabilities.sh /path/to/tuo-progetto

# Installa tutte le capability stable senza interazione
./claude-catalog/scripts/setup-capabilities.sh /path/to/tuo-progetto all
```

2.2 Flusso interattivo

Una volta avviato, lo script mostra la lista di tutte le capability disponibili con tier e descrizione:

```
╔═══════════════════════════════════════════╗
║     Claude Registry — Setup Capabilities     ║
╚═══════════════════════════════════════════╝

  Registry:  /Users/luca/dev/claude-registry
  Progetto:  /Users/luca/dev/mio-progetto

Capability disponibili:

  Num  Tier    Nome                          Descrizione
  ───  ──────  ──────────────────────────  ──────────────────────────────────────────
    1  stable  software-architect            Analyzes and designs system architectur...
    2  stable  functional-analyst            Extracts and documents functional requi...
    3  stable  developer-java-spring         Writes production-ready Java/Spring Boo... [uses: ja
    4  beta    technical-analyst             Produces structured technical analysis...
   ...

  Digita i numeri separati da spazio, oppure:
  all  — installa tutte le stable
  q    — esci

  Selezione: 1 3 6
```

2.3 Risoluzione automatica delle skill

Le **skill** sono knowledge provider condivisi tra più agenti. Non devi selezionarle manualmente: lo script le installa automaticamente come dipendenze degli agenti che hai scelto.

Esempio: selezionando `developer-java-spring`, lo script installa anche:

- `java-spring-standards` — standard Java/Spring Boot
- `testing-standards` — principi e template di test
- `rest-api-standards` — design REST e OpenAPI

```
Installazione:

  ✓ developer-java-spring [stable]
    ↳ java-spring-standards [skill — dipendenza]
    ↳ testing-standards [skill — dipendenza]
    ↳ rest-api-standards [skill — dipendenza]
```

Cosa sono le skill? Le skill sono file subagent con `model: haiku` e `tools: Read`. Non vengono invocate direttamente dall'utente — sono chiamate internamente dagli agenti per caricare standard e convenzioni (es. regole di naming, palette colori Accenture, template JUnit 5). Per il consumatore non cambia nulla: installi un agente, le sue dipendenze arrivano in automatico.

2.4 Aggiornamento automatico di settings.json

Al termine dell'installazione, lo script aggiorna (o crea) `.claude/settings.json` con le regole di permesso necessarie affinché Claude Code possa delegare ai subagent senza chiedere conferma ogni volta:

```
{
  "permissions": {
    "allow": [
      "Agent(software-architect)",
      "Agent(developer-java-spring)",
      "Agent(java-spring-standards)",
      "Agent(testing-standards)",
      "Agent(rest-api-standards)"
    ]
  }
}
```

> **Nota:** anche le skill vengono aggiunte ai permessi, perché gli agenti le invocano tramite il tool `Agent`. Se non fossero presenti, Claude Code chiederebbe conferma ad ogni invocazione di una skill.

2.5 Installazione manuale (alternativa)

Se preferisci copiare i file manualmente senza usare lo script:

```
mkdir -p .claude/agents

# Copia l'agente
cp claude-marketplace/stable/software-architect.md .claude/agents/

# Verifica le dipendenze in catalog.json, poi copia le skill
cp claude-marketplace/skills/java-spring-standards.md .claude/agents/
```

Consulta `claude-marketplace/catalog.json` per il campo `"dependencies"` di ogni agente — elenca le skill da copiare insieme all'agente.

2.6 Commit nel progetto

Una volta installate le capability, aggiungile al controllo di versione del tuo progetto:

```
cd /path/to/tuo-progetto
git add .claude/
git commit -m "add: claude capabilities (developer-java-spring, software-architect)"
```

In questo modo tutti i membri del team avranno le stesse capability disponibili automaticamente.

3. Verifica dell'installazione

3.1 Comando /agents

Apri Claude Code nella directory del tuo progetto e digita `/agents`. Vedrai la lista di tutti i subagent disponibili, inclusi quelli appena installati.

> **Cosa aspettarsi:** gli agenti appaiono nella lista con il loro nome e descrizione. Le skill sono visibili ma non devono essere invocate direttamente — vengono usate automaticamente dagli agenti che ne dipendono.

3.2 Verifica per delega automatica

Il modo più naturale di verificare è avviare un task che corrisponde alla descrizione di una capability. Claude Code delega automaticamente al subagent appropriato:

- "Puoi analizzare l'architettura di questo servizio?" → delega a `software-architect`
- "Scrivi i test per questa classe" → delega a `test-writer`
- "Fai una code review del file X" → delega a `code-reviewer`

3.3 Capability disponibili — Agenti (12)

Nome	Tier	Quando usarlo	Dipendenze skill
`software-architect`	STABLE	Analisi architetturale, ADR, trade-off	—
`functional-analyst`	STABLE	Requisiti, use case, processi di business	—
`developer-java-spring`	STABLE	Sviluppo Java/Spring Boot enterprise	java-spring-standards, testing-standards, rest-api-standards
`technical-analyst`	BETA	Debito tecnico, sicurezza, dipendenze	—
`developer-python`	BETA	Sviluppo Python/FastAPI	testing-standards
`code-reviewer`	BETA	Code review strutturata su PR o file	java-spring-standards, testing-standards, rest-api-standards
`test-writer`	BETA	Test JUnit 5, Mockito, Testcontainers, pytest	testing-standards, java-spring-standards
`debugger`	BETA	Diagnosi bug da stack trace, log e codice	—
`api-designer`	BETA	Design e review API REST, spec OpenAPI	rest-api-standards
`documentation-writer`	BETA	README, runbook, guide architetturali	—
`presentation-creator`	BETA	Slide Accenture-branded (.pptx)	accenture-branding
`document-creator`	BETA	Documenti Accenture-branded (PDF/DOCX)	accenture-branding

3.4 Capability disponibili — Skill (4)

Le skill vengono installate automaticamente come dipendenze. Non è necessario selezionarle esplicitamente con lo script.

Nome	Contenuto	Usata da
`java-spring-standards`	Struttura pacchetti, layering, testing, error handling, logging, security, Micrometer	developer-java-spring, code-reviewer, test-writer
`testing-standards`	Principi di testing, tassonomia scenari, naming, JUnit 5 / pytest / Jest template	developer-java-spring, test-writer, code-reviewer, developer-python
`rest-api-standards`	Modellazione risorse, HTTP methods, status code, RFC 7807, OpenAPI 3.1	developer-java-spring, api-designer, code-reviewer
`accenture-branding`	Palette colori, costanti python-pptx, CSS template PDF, tipografia	presentation-creator, document-creator

4. Aggiornare e personalizzare le capability

4.1 Aggiornare alla versione più recente

Per aggiornare le capability al contenuto più recente del marketplace, riesegui lo script di installazione. I file esistenti vengono sovrascritti:

```
# Aggiorna tutte le stable (sovrascrive quelle già presenti)
./claude-catalog/scripts/setup-capabilities.sh /path/to/tuo-progetto all

# Aggiorna selettivamente
./claude-catalog/scripts/setup-capabilities.sh /path/to/tuo-progetto
```

4.2 Fissare una versione specifica

Se il progetto deve rimanere su una versione specifica di una capability (es. per stabilità in produzione):

```
# Controlla il tag desiderato in catalog.json
git checkout software-architect@1.0.0 -- claude-marketplace/stable/software-architect.md

# Copia nel progetto
cp claude-marketplace/stable/software-architect.md /path/to/progetto/.claude/agents/
```

Documenta la versione fissa nel `CLAUDE.md` del tuo progetto, spiegando il motivo.

4.3 Override locale di una capability

Per aggiungere contesto specifico al progetto senza modificare la capability globale:

1. **Copia la capability con un nuovo nome**

```
cp .claude/agents/developer-java-spring.md \
   .claude/agents/developer-java-spring-payments.md
```

2. **Modifica il campo name nel frontmatter**

```
name: developer-java-spring-payments
```

3. **Aggiungi contesto specifico in fondo al system prompt**

```
## Project context (Payments Service)
- Package root: com.acme.payments
```

```
- Usa la libreria interna: com.acme.commons.audit.AuditLogger
- Tutti gli importi monetari usano BigDecimal, mai double o float
```

Regola d'oro: gli override locali devono essere sottili. L'obiettivo è aggiungere contesto di progetto, non riscrivere la capability. Se un override cresce molto, valuta se il contenuto aggiunto dovrebbe diventare parte della capability globale nel registry.

PARTE 2

Contribuire al Registry

5. Struttura del repository

```
claude-registry/
  claude-catalog/                ← sorgente (sviluppo e review)
    agents/                        subagent .md con YAML frontmatter
    skills/                        knowledge provider riutilizzabili
    examples/                      invocazioni di esempio
    evals/                         scenari di validazione
    hooks/                         script e configurazioni per hook
    settings/                      reference settings.json
    mcp/                           esempi di configurazione MCP server
    policies/                      convenzioni residue (da migrare a skill)
    templates/                     template di output (ADR, report, API contract)
    docs/                          documentazione interna per i contributori
    scripts/
      setup-capabilities.sh        installa capability in un progetto
      new-capability.sh            scaffolda nuova capability o skill
    how-to-write-a-capability.md
    CONTRIBUTING.md
    GOVERNANCE.md
    CHANGELOG.md

  claude-marketplace/            ← distribuzione (solo capability approvate)
    stable/                        capability production-ready
    beta/                          capability nuove o sperimentali
    skills/                        skill condivise (dipendenze automatiche)
    catalog.json                   manifest con versioni, metadati e dipendenze

  guida-operativa.pdf            ← questo documento
  pitch-claude-registry.pptx
```

5.1 Separazione catalog / marketplace

La directory `claude-catalog/` è dove avviene lo sviluppo: si modificano i file, si apre la PR, si fa la review. La directory `claude-marketplace/` contiene solo le capability approvate e pubblicate — non si modifica direttamente. Il passaggio avviene tramite il processo di release (sezione 11).

> **Regola:** non modificare mai direttamente i file in `claude-marketplace/`. Tutte le modifiche passano da `claude-catalog/` e dal processo di PR + publish.

6. Creare un nuovo agente

6.1 Scaffolding con new-capability.sh

Lo script crea automaticamente tutti i file necessari e il branch git:

```
./claude-catalog/scripts/new-capability.sh api-security-reviewer
```

Lo script crea:

- `claude-catalog/agents/api-security-reviewer.md` — file capability con template
- `claude-catalog/examples/api-security-reviewer-example.md` — esempio di invocazione
- `claude-catalog/evals/api-security-reviewer-eval.md` — scenari di validazione
- Branch git `add/api-security-reviewer` — pronto per il lavoro
- Entry in `CHANGELOG.md` sotto `[Unreleased]`

6.2 Formato del file capability

Ogni capability è un file Markdown con frontmatter YAML e system prompt:

```
---
name: api-security-reviewer
description: >
  Use when reviewing REST API endpoints for security vulnerabilities:
  authentication gaps, authorization flaws, injection risks, sensitive
  data exposure, and missing rate limiting. Does not write fix code —
  delegates to developer subagents for implementation.
tools: Read, Grep, Glob
model: sonnet
color: red
---

## Role

You are a senior API security engineer. Your role is to identify
security vulnerabilities in REST API designs and implementations...

## What you always do

- Read all relevant endpoint code before reporting findings
- Cite exact file:line evidence for every finding
- Rate severity: Critical / High / Medium / Low
- Provide remediation guidance for each finding

## What you never do

- Guess vulnerabilities without reading the actual code
- Write implementation fixes (describe them, delegate to developer)

## Output format
```

```
...

## Quality self-check before responding

...
```

6.3 Campi frontmatter obbligatori

Campo	Tipo	Note
`name`	string	Lowercase, solo lettere e trattini. Deve coincidere col nome file.
`description`	string	Inizia con "Use when" o "Use for". Campo più critico: controlla la delega automatica.
`tools`	string	Lista separata da virgole. Usa il set minimo necessario.
`model`	string	Usa `sonnet` come default. `opus` solo se giustificato.

6.4 Profili tool standard

Profilo	Tool	Quando usarlo
Read-only analyst	`Read, Grep, Glob`	Ruoli di analisi pura (architect, analyst)
Read + report	`Read, Grep, Glob, Write`	Analisi con output scritto (ADR, report)
Full developer	`Read, Edit, Write, Bash, Grep, Glob`	Sviluppo attivo di codice
Research	`Read, Grep, Glob, WebFetch`	Architettura con lookup web

6.5 Come usare le skill esistenti in un agente

Se il tuo agente deve applicare standard già definiti in una skill, aggiungili nella sezione `## Skills` del system prompt invece di copiare il contenuto:

```
## Skills

Before starting any task, invoke the following skills to load shared standards:

- `java-spring-standards` — package structure, layering rules, dependency
  injection, testing patterns, error handling, logging, security.
  Invoke with: "Provide all standards relevant to: [task description]"

- `testing-standards` — testing principles, scenario taxonomy, method
  naming, JUnit 5 + Mockito + Testcontainers templates.
  Invoke when the task involves writing or reviewing tests.

Apply the returned standards as non-negotiable guidelines in your output.
```

> **Principio:** non copiare standard da una skill nel system prompt di un agente. Usa il meccanismo `## Skills`. Questo garantisce che tutti gli agenti usino la stessa versione degli standard e che gli aggiornamenti alle skill si propaghino automaticamente a tutti gli agenti che le usano.

Quando aggiungi una skill come dipendenza di un agente, aggiorna anche il file `catalog.json`:

```
{
  "name": "il-tuo-agente",
  "dependencies": ["java-spring-standards", "testing-standards"],
  ...
}
```

7. Creare una nuova skill

7.1 Agenti vs. Skill — quando usare cosa

	Agente	Skill
Scopo	Esegue un ruolo: analizza, scrive codice, produce output	Fornisce conoscenza: standard, convenzioni, regole
Model	`sonnet` (default)	`haiku` obbligatorio
Tool	Set appropriato al ruolo	Solo `Read`
Invocato da	Claude (delega automatica) o utente (`/agents`)	Solo altri agenti tramite `Agent` tool
Azioni	Sì — modifica file, esegue comandi, produce output	No — restituisce solo conoscenza
Location	`agents/`	`skills/`

Crea una skill quando la stessa conoscenza di dominio (standard di codice, regole di design, costanti di brand) compare o comparirebbe in due o più system prompt di agenti diversi.

7.2 Scaffolding della skill

```
./claude-catalog/scripts/new-capability.sh --type skill graphql-standards
```

Lo script crea il file in `claude-catalog/skills/graphql-standards.md` con il template corretto e il branch `add/graphql-standards`.

7.3 Formato del file skill

```
---
name: graphql-standards
description: >
  Use to retrieve GraphQL API design standards: schema design,
  resolver patterns, error handling, pagination (Relay spec),
  and security rules (query depth limiting, introspection).
tools: Read
model: haiku
color: cyan
---

## Role

You are a knowledge provider for GraphQL API design standards.
```

```
When invoked, return the complete and authoritative standards
relevant to the caller's task.

Do not take any actions. Do not write or modify files.
Return knowledge only.

---

## Schema Design

[Regole e convenzioni per la progettazione dello schema]

---

## Resolver Patterns

[Pattern obbligatori per i resolver]

---

## Error Handling

[Standard per gli errori GraphQL]
```

7.4 Vincoli obbligatori per le skill

Vincolo	Valore obbligatorio	Motivo
`model`	`haiku`	Le skill sono solo recupero conoscenza, non ragionamento profondo
`tools`	`Read` (solo)	Le skill non modificano file, non eseguono comandi, non delegano
Sezione `## Skills`	Vietata	Le skill sono nodi foglia — non possono invocare altre skill
Tool `Agent`	Vietato	Le skill non possono delegare a subagent

7.5 Aggiornare catalog.json

Dopo aver creato la skill, aggiorna `claude-marketplace/catalog.json` in due punti:

1. Aggiunta entry della skill:

```
{
  "name": "graphql-standards",
  "version": "1.0.0",
  "type": "skill",
  "tier": "skill",
  "status": "active",
  "description": "Use to retrieve GraphQL API design standards...",
  "file": "skills/graphql-standards.md",
  "dependencies": [],
  "tools": ["Read"],
  "model": "haiku",
  "tags": ["graphql", "api", "standards", "skill"],
  "published": "2026-04-20",
  "changelog": "Initial release"
}
```

2. Aggiunta alla lista dependencies degli agenti che la usano:

```
{
  "name": "api-designer",
  "dependencies": ["rest-api-standards", "graphql-standards"],
  ...
}
```

> **Importante:** il campo `"dependencies"` in `catalog.json` è quello che `setup-capabilities.sh` usa per installare automaticamente le skill. Se dimentichi di aggiornarlo, i consumatori non riceveranno la skill

quando installano l'agente.

7.6 Aggiungere la sezione Skills all'agente

Ogni agente che usa la nuova skill deve avere la skill elencata nella propria sezione `## Skills` nel system prompt (vedi sezione 6.5 per il formato).

8. Scrivere il system prompt

8.1 Struttura consigliata

Il corpo del file `.md` (dopo il frontmatter) è il system prompt. Sostituisce il prompt di default di Claude Code quando il subagent è attivo.

```
## Role
Un paragrafo: chi è il subagent, su cosa è autorevole.

## Skills                        ← solo se il subagent usa skill
Istruzioni per invocare le skill necessarie.

## What you always do
Lista puntata di comportamenti obbligatori in ogni interazione.

## What you never do
Proibizioni esplicite con motivazione se non ovvia.

## Output format
Formato esatto dell'output. Sii specifico: nomi sezioni, colonne tabelle.

## Quality self-check before responding
Lista di controlli che il subagent deve fare prima di rispondere.
```

8.2 Regole per il campo description

Il campo `description` nel frontmatter è il campo più critico: Claude Code lo usa per decidere quando delegare automaticamente a questo subagent.

Stile	Esempio	Giudizio
Vago	"Helps with software architecture."	Non usare
Technology-first	"Works with Spring Boot applications."	Non usare
Trigger preciso	"Use when analyzing or designing system architecture, evaluating technology choices, writing ADRs, or reasoning about trade-offs across security, performance, scalability, and cost."	Corretto

8.3 Sei regole per un buon system prompt

1. **Sii opinionato.** Un prompt generico produce output generico. Specifica framework, formati, standard con nome.
2. **Definisci il formato di output esattamente.** Se deve produrre un ADR, definisci la struttura esatta. Non lasciare il formato all'interpretazione.

3. **Includi completeness check.** Di' al subagent cosa verificare prima di rispondere.
4. **Nomina i framework che segue.** "Usa il modello C4" è meglio di "descrivi la struttura del sistema".
5. **Specifica cosa delega.** "Non scrivere codice implementativo. Descrivilo e indica quale developer subagent deve implementarlo."
6. **Definisci l'escalation.** Di' al subagent quando chiedere più contesto invece di procedere su assunzioni.

9. Testare localmente

Prima di aprire una PR, verifica che la capability si comporti correttamente con scenari reali.

9.1 Installazione locale temporanea

```
# Copia la capability nel progetto di test
cp claude-catalog/agents/api-security-reviewer.md \
   /path/to/test-project/.claude/agents/

# Se la capability dipende da skill, copia anche quelle
cp claude-catalog/skills/rest-api-standards.md \
   /path/to/test-project/.claude/agents/

# Apri Claude Code nel progetto di test
cd /path/to/test-project
claude
```

9.2 Eseguire gli eval

Gli scenari di eval sono in `claude-catalog/evals/nome-capability-eval.md`. Ogni scenario specifica:

- **Input context**: i file o il contesto da fornire alla sessione
- **User prompt**: il prompt esatto da inviare
- **Expected output characteristics**: cosa deve apparire nell'output
- **Must NOT contain**: anti-pattern da verificare che non compaiano

```
## Eval-001: Happy path

Input context: un controller REST con endpoint non autenticato

User prompt: "Analizza la sicurezza degli endpoint in UserController.java"

Expected output characteristics:
- Identifica l'endpoint non autenticato come finding
- Assegna severity High o Critical
- Cita file:riga come evidence
- Fornisce remediation con @PreAuthorize o SecurityConfig

Must NOT contain:
- Raccomandazioni senza evidence
- Codice implementativo
```

9.3 Checklist pre-PR

Controllo	Per agenti	Per skill
`name` coincide col filename	Obbligatorio	Obbligatorio
`description` inizia con "Use when" / "Use for"	Obbligatorio	Inizia con "Use to retrieve"
Almeno un esempio in `examples/`	Obbligatorio	Consigliato
Almeno due scenari in `evals/`	Obbligatorio	Consigliato
`model: haiku`	—	Obbligatorio
`tools: Read` (solo)	—	Obbligatorio
Nessuna sezione `## Skills`	—	Obbligatorio
`CHANGELOG.md` aggiornato sotto `[Unreleased]`	Obbligatorio	Obbligatorio
`catalog.json` aggiornato	Obbligatorio	Obbligatorio (+ dipendenze agenti)

10. Aprire una PR e gestire la review

10.1 Flusso standard per una nuova capability

```
# Lo script new-capability.sh crea già il branch — ma se stai lavorando manualmente:
git checkout -b add/api-security-reviewer

# Lavora sui file
$EDITOR claude-catalog/agents/api-security-reviewer.md
$EDITOR claude-catalog/examples/api-security-reviewer-example.md
$EDITOR claude-catalog/evals/api-security-reviewer-eval.md
# Aggiorna CHANGELOG.md sotto [Unreleased]
# Aggiorna catalog.json se necessario

# Commit e push
git add claude-catalog/
git commit -m "add: api-security-reviewer capability (draft)"
git push origin add/api-security-reviewer

# Apri la PR
gh pr create \
  --title "add: api-security-reviewer" \
  --body "Aggiunge il subagent api-security-reviewer per review di sicurezza REST"
```

10.2 Flusso per una nuova skill

```
git checkout -b add/graphql-standards

# Scrivi la skill
$EDITOR claude-catalog/skills/graphql-standards.md
# Aggiorna catalog.json: aggiungi entry skill + dipendenze agenti
# Aggiorna ## Skills section degli agenti che la usano
# Aggiorna CHANGELOG.md

git add claude-catalog/ claude-marketplace/catalog.json
git commit -m "add: graphql-standards skill"
git push origin add/graphql-standards
gh pr create --title "add: graphql-standards skill"
```

10.3 Breaking changes

Una modifica è **breaking** se cambia il campo `name` o `description` nel frontmatter. Questi campi controllano come Claude Code decide di invocare il subagent: cambiarli altera il comportamento di tutti i progetti che usano la capability.

- Indica chiaramente "BREAKING CHANGE" nel titolo e nella descrizione della PR
- Esegui un bump della versione major in `catalog.json`
- Aggiungi una nota di migrazione in `CHANGELOG.md` che spiega cosa devono aggiornare i team

10.4 SLA di review

Tipo	SLA
Review PR (nuova capability o modifica)	2 giorni lavorativi
Triage bug su capability pubblicata	5 giorni lavorativi
Rimozione capability deprecata	90 giorni dalla notice di deprecazione

11. Pubblicare nel marketplace

11.1 Lifecycle di una capability

```
draft → review → approved → released → published → deprecated
```

Stato	Significato	Dove si trova
`draft`	Lavoro in corso su feature branch	`claude-catalog/` (branch)
`review`	PR aperta, review in corso	(PR)
`approved`	PR mergiata su main	`claude-catalog/` (main)
`released`	Git tag applicato (`name@x.y.z`)	Git tag
`published`	Copiata nel tier del marketplace	`claude-marketplace/stable/`, `beta/` o `skills/`
`deprecated`	Non più raccomandata, mantenuta per compatibilità	Marketplace con notice di deprecazione

11.2 Applicare il git tag e copiare nel marketplace

```
# Dopo il merge su main
git checkout main
git pull

# Applica il tag SemVer
git tag api-security-reviewer@0.1.0
git push origin api-security-reviewer@0.1.0

# Copia nel tier appropriato (beta per nuove capability)
cp claude-catalog/agents/api-security-reviewer.md \
   claude-marketplace/beta/api-security-reviewer.md

# Per una skill:
cp claude-catalog/skills/graphql-standards.md \
   claude-marketplace/skills/graphql-standards.md

# Commit e push del marketplace
git add claude-marketplace/
git commit -m "publish: api-security-reviewer@0.1.0 (beta)"
git push origin main
```

11.3 Promozione da beta a stable

Una capability può essere promossa da `beta` a `stable` quando soddisfa entrambi i requisiti:

- Usata in almeno **due progetti distinti**
- Almeno **30 giorni** senza issue critiche aperte

```
# Promozione a stable
cp claude-marketplace/beta/api-security-reviewer.md \
   claude-marketplace/stable/api-security-reviewer.md

# Aggiorna tier in catalog.json
# "tier": "beta"  →  "tier": "stable"

git add claude-marketplace/ catalog.json
git commit -m "promote: api-security-reviewer beta → stable"
```

11.4 Versioning SemVer

- **PATCH** (x.y.Z): correzioni di bug nel prompt, senza cambi comportamentali
- **MINOR** (x.Y.z): nuovi comportamenti, nuovi output format, nuove skill aggiunge
- **MAJOR** (X.y.z): breaking change su `name` o `description`

Governance

Ruoli

Ruolo	Responsabilità
Capability Author	Scrive nuovi agenti o skill; fornisce esempi ed eval
Capability Reviewer	Rivede le PR con la review-checklist; approva o richiede modifiche
Catalog Maintainer	Mergia le PR approvate, gestisce i git tag, esegue il processo di publish
Consumer	Usa le capability pubblicate; segnala bug e gap tramite issue

> Un membro del team può ricoprire più ruoli. Non è richiesto un team dedicato — il processo scala da un singolo maintainer a molti.

Cosa non fare

- Non inserire credenziali, API key o segreti nel repository
- Non modificare direttamente i file in `claude-marketplace/`
- Non bypassare la review anche per "fix piccoli" — una modifica di una riga nel prompt può alterare significativamente il comportamento
- Non versionare le capability nel nome del file — usa i git tag
- Non copiare conoscenza già definita in una skill direttamente nel system prompt di un agente — usa il meccanismo `## Skills`

Risorse collegate

Risorsa	Percorso
Come scrivere una capability	`claude-catalog/how-to-write-a-capability.md`
Checklist review PR	`claude-catalog/review-checklist.md`
Processo di release	`claude-catalog/release-process.md`
Governance dettagliata	`claude-catalog/GOVERNANCE.md`
Changelog	`claude-catalog/CHANGELOG.md`
Manifest capability	`claude-marketplace/catalog.json`

Claude Registry

Versione 2.0 — Aprile 2026 — Accenture Internal

12 Agenti · 4 Skill · Skill Layer Architecture